

LESSON 2

Arrays & Two Pointers

The #1 most common interview topic. Master every pattern.

- 2.1 Brute Force First Rule
- 2.2 Opposite Direction Two Pointers
- 2.3 Same Direction (Slow/Fast) Two Pointers
- 2.4 Sliding Window Pattern
- 2.5 Prefix Sum
- 2.6 Merge Two Sorted Arrays
- 2.7 Dutch National Flag (3-way partition)
- 2.8 Practice Problems with Solutions

2.1 The Brute Force First Rule

ALWAYS start with the simplest solution. Say it out loud. Then optimize. This shows your thinking process and earns you points even before writing optimal code.

```
# Problem: Find two numbers that add up to target
# Input: nums = [2, 7, 11, 15], target = 9
# Output: [0, 1] (nums[0] + nums[1] = 2 + 7 = 9)

# STEP 1: Brute Force - check every pair -  $O(n^2)$ 
def two_sum_brute(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []

# STEP 2: Optimize with Hash Map -  $O(n)$ 
def two_sum(nums, target):
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

# SAY TO INTERVIEWER:
# "I went from  $O(n^2)$  to  $O(n)$  by trading space for time -
# using a hash map to avoid the inner loop."
```

Trace: nums = [2, 7, 11, 15], target = 9

i=0, num=2: complement = 9-2 = 7
Is 7 in seen? seen={} -> NO
Store: seen = {2: 0}

i=1, num=7: complement = 9-7 = 2
Is 2 in seen? seen={2: 0} -> YES!
Return [seen[2], 1] = [0, 1]

Check: nums[0] + nums[1] = 2 + 7 = 9 Correct!

2.2 Opposite Direction Two Pointers

One pointer at start, one at end. Move toward each other. Works on **sorted arrays** and symmetry problems.

Example 1: Reverse an Array

```
def reverse_array(nums):
    left = 0
    right = len(nums) - 1
    while left < right:
        nums[left], nums[right] = nums[right], nums[left] # swap
        left += 1
        right -= 1
    return nums
```

```
[1, 2, 3, 4, 5]
L           R   swap 1,5 -> [5, 2, 3, 4, 1]
  L       R   swap 2,4 -> [5, 4, 3, 2, 1]
    LR       left meets right -> STOP
Result: [5, 4, 3, 2, 1]
```

Example 2: Palindrome Check

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

# With cleaning (ignore spaces, punctuation, case)
def is_valid_palindrome(s):
    cleaned = "".join(c.lower() for c in s if c.isalnum())
    return cleaned == cleaned[::-1]

is_valid_palindrome("A man, a plan, a canal: Panama") # True
```

```
"racecar"
r a c e c a r
L           R   r==r move inward
  L       R   a==a move inward
    L   R   c==c move inward
      LR   STOP -> True

"hello"
h e l l o
L           R   h != o -> return False immediately
```

Example 3: Two Sum on Sorted Array

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        current_sum = nums[left] + nums[right]
        if current_sum == target:
            return [left, right]
        elif current_sum < target:
            left += 1      # need bigger sum -> move left forward
        else:
            right -= 1     # need smaller sum -> move right back
    return []
```

```
nums = [1, 3, 5, 7, 9], target = 8
```

```
left=0, right=4: 1+9=10 > 8 -> right=3
left=0, right=3: 1+7=8 = 8 -> return [0, 3]
```

WHY IT WORKS: Array is sorted.

Sum too big? -> shrink from right (smaller numbers)

Sum too small? -> grow from left (bigger numbers)

We never miss a valid pair.

Example 4: Container With Most Water (LeetCode #11)

```
def max_water(heights):
    left, right = 0, len(heights) - 1
    max_area = 0

    while left < right:
        width = right - left
        height = min(heights[left], heights[right])
        area = width * height
        max_area = max(max_area, area)

        # Move the SHORTER wall (to try finding taller)
        if heights[left] < heights[right]:
            left += 1
        else:
            right -= 1

    return max_area
```

```
heights = [1, 8, 6, 2, 5, 4, 8, 3, 7]
```

```
L=0(h=1), R=8(h=7): area=min(1,7)*8=8. Left shorter,move left
L=1(h=8), R=8(h=7): area=min(8,7)*7=49. Right shorter,move right
L=1(h=8), R=7(h=3): area=min(8,3)*6=18. Right shorter,move right
L=1(h=8), R=6(h=8): area=min(8,8)*5=40. Equal,move either
... continues ...
Answer: 49
```

KEY INSIGHT: Moving shorter wall might find taller one.

Moving taller wall can only decrease width, never increase height.

2.3 Same Direction Two Pointers (Slow/Fast)

Both start at beginning. **Fast** scans ahead, **slow** builds result. The idea: fast explores, slow records.

Example 1: Remove Duplicates from Sorted Array

```
# LeetCode #26 - Very commonly asked
# Input:  [1, 1, 2, 3, 3, 4]
# Output: 4, array becomes [1, 2, 3, 4, _, _]

def remove_duplicates(nums):
    if len(nums) == 0:
        return 0

    slow = 0

    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]: # found new unique element
            slow += 1                # advance slow
            nums[slow] = nums[fast]   # write it

    return slow + 1 # +1 because slow is index, we want count
```

```
[1, 1, 2, 3, 3, 4]
s f      1==1 same    -> skip
s   f    2!=1 DIFF    -> slow=1, write 2 -> [1,2,...]
s     f  3!=2 DIFF    -> slow=2, write 3 -> [1,2,3,...]
s      f 3==3 same    -> skip
s       f 4!=3 DIFF -> slow=3, write 4 -> [1,2,3,4,_,_]

Return slow+1 = 4 (4 unique elements)
```

Example 2: Remove Element (LeetCode #27)

```
# Remove all occurrences of val, return new length
def remove_element(nums, val):
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != val: # found a keeper
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

```
nums = [3, 2, 2, 3, 4], val = 3
s
f  nums[f]=3, equals val -> SKIP
s f  nums[f]=2, not val -> write nums[0]=2, slow=1
s f  nums[f]=2, not val -> write nums[1]=2, slow=2
s f  nums[f]=3, equals val -> SKIP
s   f  nums[f]=4, not val -> write nums[2]=4, slow=3

Return 3. First 3 elements: [2, 2, 4]
```

Example 3: Move Zeroes (LeetCode #283)

```
# Move all zeroes to end, maintain order of non-zero elements
# Input:  [0, 1, 0, 3, 12]
# Output: [1, 3, 12, 0, 0]
```

```
def move_zeroes(nums):
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
```

```
[0, 1, 0, 3, 12]
s f   nums[f]=1 != 0 -> swap -> [1, 0, 0, 3, 12], slow=1
s f   nums[f]=0 -> skip
s f   nums[f]=3 != 0 -> swap -> [1, 3, 0, 0, 12], slow=2
s f   nums[f]=12 != 0 -> swap -> [1, 3, 12, 0, 0]
Done!
```

Example 4: Sort Colors / Dutch National Flag (LeetCode #75)

```
# Sort array containing only 0s, 1s, and 2s in ONE pass
# Uses THREE pointers
```

```
def sort_colors(nums):
    low = 0          # boundary for 0s
    mid = 0          # current element being examined
    high = len(nums) - 1 # boundary for 2s

    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1 # 1 is already in right place
        else: # nums[mid] == 2
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
            # DON'T move mid - need to check swapped element

# Mental model:
# [ 0s | 1s | unsorted | 2s ]
#   low  mid          high
```

[2, 0, 1, 2, 0, 1] L=0, M=0, H=5

M=0: nums[0]=2 -> swap with H -> [1,0,1,2,0,2] H=4

M=0: nums[0]=1 -> mid++ -> M=1

M=1: nums[1]=0 -> swap with L -> [0,1,1,2,0,2] L=1,M=2

M=2: nums[2]=1 -> mid++ -> M=3

M=3: nums[3]=2 -> swap with H -> [0,1,1,0,2,2] H=3

M=3: nums[3]=0 -> swap with L -> [0,0,1,1,2,2] L=2,M=4

M=4 > H=3 -> STOP

Result: [0, 0, 1, 1, 2, 2]

2.4 Sliding Window Pattern

For problems about **contiguous subarrays**. The window slides across the array.

Fixed Window: Max Sum of K Elements

```
def max_sum_subarray(nums, k):
    window_sum = sum(nums[:k])    # first window
    max_sum = window_sum

    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]    # add new, remove old
        max_sum = max(max_sum, window_sum)

    return max_sum
```

[2, 1, 5, 1, 3, 2] k=3

[2, 1, 5]	sum=8
[1, 5, 1]	sum=8-2+1=7
[5, 1, 3]	sum=7-1+3=9 <- max!
[1, 3, 2]	sum=9-5+2=6

Answer: 9

KEY: Instead of recalculating sum ($O(k)$ each time), just add new element and remove old one ($O(1)$).

Variable Window: Smallest Subarray with Sum $\geq$ Target

```
def min_subarray_sum(nums, target):
    left = 0
    current_sum = 0
    min_length = float("inf")

    for right in range(len(nums)):
        current_sum += nums[right]    # expand window

        while current_sum >= target:    # try to shrink
            min_length = min(min_length, right - left + 1)
            current_sum -= nums[left]
            left += 1

    return min_length if min_length != float("inf") else 0
```



```

nums = [2, 3, 1, 2, 4, 3], target = 7

right=0: sum=2
right=1: sum=5
right=2: sum=6
right=3: sum=8 >= 7! len=4. Shrink: sum=8-2=6, left=1
right=4: sum=10 >= 7! len=4. Shrink: sum=10-3=7 >=7! len=3.
    Shrink: sum=7-1=6, left=3
right=5: sum=9 >= 7! len=3. Shrink: sum=9-2=7 >=7! len=2!
    Shrink: sum=7-4=3, left=5

Answer: 2 (subarray [4, 3])

```

2.5 Prefix Sum

```

# Prefix sum lets you find sum of ANY subarray in O(1)

# Build prefix sum array
nums = [3, 1, 4, 1, 5]
prefix = [0]
for num in nums:
    prefix.append(prefix[-1] + num)

# prefix = [0, 3, 4, 8, 9, 14]
#           ^  ^  ^  ^  ^  ^
#           sum sum sum sum sum
#           of0 of1 of2 of3 of4
#           elts elt elts elts elts

# Sum from index i to j (inclusive):
# sum(i, j) = prefix[j+1] - prefix[i]

# Example: sum from index 1 to 3
# = prefix[4] - prefix[1] = 9 - 3 = 6
# Check: nums[1]+nums[2]+nums[3] = 1+4+1 = 6 Correct!

```

2.6 Merge Two Sorted Arrays

```
def merge_sorted(a, b):
    i, j = 0, 0
    result = []

    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1

    result.extend(a[i:]) # remaining elements
    result.extend(b[j:])
    return result
```

```
a = [1, 3, 5, 7], b = [2, 4, 6]
```

```
i=0,j=0: 1<2 -> take 1 from a    result=[1]
i=1,j=0: 3>2 -> take 2 from b    result=[1,2]
i=1,j=1: 3<4 -> take 3 from a    result=[1,2,3]
i=2,j=1: 5>4 -> take 4 from b    result=[1,2,3,4]
i=2,j=2: 5<6 -> take 5 from a    result=[1,2,3,4,5]
i=3,j=2: 7>6 -> take 6 from b    result=[1,2,3,4,5,6]
j done -> take rest from a      result=[1,2,3,4,5,6,7]
```

2.7 Practice Problems with Full Solutions

Problem 1: Find Pair with Given Difference

```
# Given sorted array, find pair where a[j] - a[i] = target
def find_pair_diff(nums, target):
    i, j = 0, 1
    while i < len(nums) and j < len(nums):
        diff = nums[j] - nums[i]
        if i == j:                # can't use same element
            j += 1
        elif diff == target:
            return [i, j]
        elif diff < target:
            j += 1                # need bigger difference
        else:
            i += 1                # need smaller difference
    return []

# find_pair_diff([1, 3, 5, 7, 9], 4) -> [0, 2] (5-1=4)
```

Problem 2: Maximum Product of Two Elements

```
# Return max product of any two elements
def max_product(nums):
    nums.sort()
    # Could be: two largest positives OR two most negative
    return max(nums[0] * nums[1], nums[-1] * nums[-2])

# max_product([-10, -3, 1, 2, 5]) -> 30 (-10 * -3 = 30)
```

Problem 3: Three Sum (LeetCode #15 - Very Popular)

```

# Find all unique triplets that sum to 0
def three_sum(nums):
    nums.sort()
    result = []

    for i in range(len(nums) - 2):
        # Skip duplicates for i
        if i > 0 and nums[i] == nums[i-1]:
            continue

        left = i + 1
        right = len(nums) - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]

            if total == 0:
                result.append([nums[i], nums[left], nums[right]])
                # Skip duplicates
                while left < right and nums[left] == nums[left+1]:
                    left += 1
                while left < right and nums[right] == nums[right-1]:
                    right -= 1
                left += 1
                right -= 1
            elif total < 0:
                left += 1
            else:
                right -= 1

        return result

# three_sum([-1,0,1,2,-1,-4]) -> [[-1,-1,2], [-1,0,1]]

```

>> Three Sum = Fix one number + Two Sum on the rest.
Sort first, then for each `nums[i]`, use two pointers on remaining.

Problem 4: Maximum Subarray Sum (Kadane's Algorithm)

```

# Find contiguous subarray with largest sum
# Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
# Output: 6 (subarray [4, -1, 2, 1])

def max_subarray(nums):
    current_sum = nums[0]
    max_sum = nums[0]

    for i in range(1, len(nums)):
        # Either extend current subarray or start new one
        current_sum = max(nums[i], current_sum + nums[i])
        max_sum = max(max_sum, current_sum)

    return max_sum

```

```
[-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

```
i=0: curr=-2, max=-2
i=1: curr=max(1, -2+1)=1, max=1
i=2: curr=max(-3, 1-3)=-2, max=1
i=3: curr=max(4, -2+4)=4, max=4
i=4: curr=max(-1, 4-1)=3, max=4
i=5: curr=max(2, 3+2)=5, max=5
i=6: curr=max(1, 5+1)=6, max=6 <- answer!
i=7: curr=max(-5, 6-5)=1, max=6
i=8: curr=max(4, 1+4)=5, max=6
```

KEY INSIGHT: At each position, decide:

"Should I extend the current subarray, or start fresh here?"

Two Pointer Pattern Cheat Sheet

Pattern	Setup	When to Use	Classic Problems
Opposite Direction	left=0 right=end	Sorted array, pairs, symmetry	Two Sum sorted, Palindrome, Container Water, 3Sum
Slow/Fast Same Dir	slow=0 fast loops	Remove/filter in-place, partitioning	Remove Dups, Move Zeroes, Remove Element, Sort Colors
Two Arrays	i in A j in B	Merging, comparing sorted data	Merge Sorted, Intersection
Fixed Window	window of size k	Subarray of exact size k	Max Sum of K, Avg of K
Variable Window	left,right expand/shrink	Longest/shortest subarray with condition	Min Size Subarray, Longest Unique Substring